

DIP HW3_3

313512009 施丞蔚

1. Please use a self-designed Lowpass Gaussian Filter Kernels to cancel the shaded pattern noise of the image, 'checkerboard1024-shaded.tif'. Please describe the your lowpass Gaussian filter kernels, shading noise pattern as Fig. 3.42 (b), final corrected image without shading noise pattern as Fig. 3.42 (c) and print out the source code?(40)

```
import argparse
from pathlib import Path
import numpy as np, cv2, matplotlib.pyplot as plt

def read_gray(path: Path) -> np.ndarray:
    if not path.exists():
        raise FileNotFoundError(f"[ERROR] File not found: {path}")
    img = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if img is None:
        try:
            import imageio.v2 as imageio
            img = imageio.imread(path)
            if img.ndim == 3: img = cv2.cvtColor(img,
cv2.COLOR_RGB2GRAY)
        except Exception:
            from PIL import Image
            img = np.array(Image.open(path).convert("L"))
    if img.ndim == 3: img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return img.astype(np.float64)

def gaussian_params(h, w, K):
    sigma = min(h, w)/K
    ksize = int(2*np.ceil(3*sigma)+1)
    return int(ksize), float(sigma)

def blur(img, ksize, sigma):
    if 0 < ksize <= 1001 and ksize % 2 == 1:
        return cv2.GaussianBlur(img, (ksize, ksize), sigma, sigma,
borderType=cv2.BORDER_REFLECT)
    return cv2.GaussianBlur(img, (0, 0), sigma, sigma,
borderType=cv2.BORDER_REFLECT)

def flatfield(img, shade, eps=1e-6):
    shade_n = shade/(shade.mean()+eps)
```

```

    return img/(shade_n+eps)

def to_u8(x):
    x = x - x.min()
    if x.max()>0: x = x/x.max()
    return (x*255.0+0.5).astype(np.uint8)

def main():
    ap = argparse.ArgumentParser(description="Q1: Gaussian lowpass
shading correction on checkerboard")
    ap.add_argument("--img", default="./checkerboard1024-shaded.tif")
    ap.add_argument("--out", default="./outputs_q1")
    ap.add_argument("--K", type=float, default=18.0, help="sigma =
min(H,W)/K (bigger blur when smaller K)")
    ap.add_argument("--no-show", action="store_true")
    args = ap.parse_args()

    img = read_gray(Path(args.img))
    h, w = img.shape
    ksize, sigma = gaussian_params(h, w, args.K)
    shade = blur(img, ksize, sigma)
    corr = flatfield(img, shade)

    out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
    base = Path(args.img).stem
    p_shade = out/f"{base}_shade.png"
    p_corr = out/f"{base}_corrected.png"
    p_panel = out/f"{base}_panel.png"

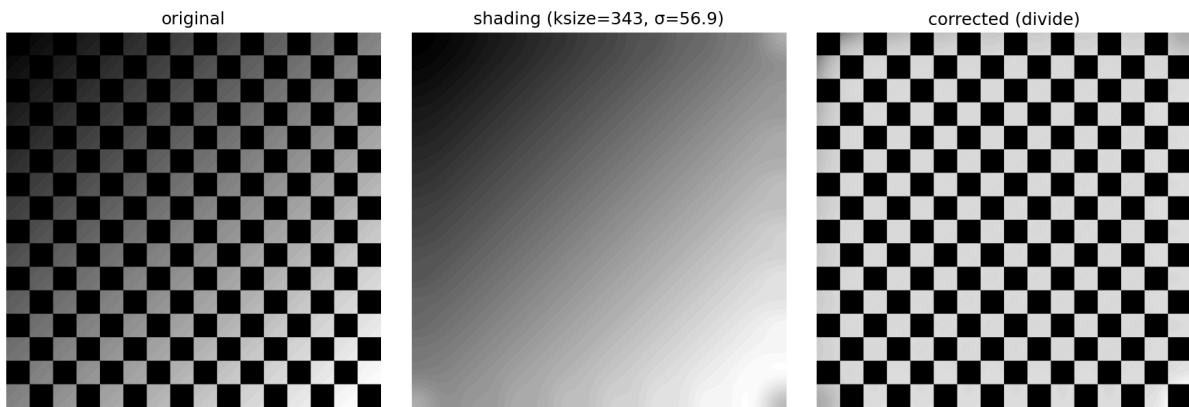
    cv2.imwrite(str(p_shade), to_u8(shade))
    cv2.imwrite(str(p_corr), to_u8(corr))

    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1); plt.imshow(to_u8(img), cmap="gray");
plt.title("original"); plt.axis("off")
    plt.subplot(1,3,2); plt.imshow(to_u8(shade), cmap="gray");
plt.title(f"shading (ksize={ksize}, σ={sigma:.1f})"); plt.axis("off")
    plt.subplot(1,3,3); plt.imshow(to_u8(corr), cmap="gray");
plt.title("corrected (divide)"); plt.axis("off")
    plt.tight_layout(); plt.savefig(p_panel, dpi=150,
bbox_inches="tight")
    if not args.no_show: plt.show()
    else: plt.close()

```

```
print(f"[Q1 OK] size={h}x{w} kernel≈{ksize} sigma={sigma:.2f}")
print(f" shading → {p_shade}")
print(f" corrected → {p_corr}")
print(f" panel → {p_panel}")

if __name__ == "__main__":
    main()
```



2. Repeat (1) steps in the image 'N1.bmp' (40)

```
import argparse
from pathlib import Path
import numpy as np, cv2, matplotlib.pyplot as plt

def read_gray(path: Path) -> np.ndarray:
    if not path.exists():
        raise FileNotFoundError(f"[ERROR] File not found: {path}")
    img = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if img is None:
        try:
            import imageio.v2 as imageio
            img = imageio.imread(path)
            if img.ndim == 3: img = cv2.cvtColor(img,
cv2.COLOR_RGB2GRAY)
        except Exception:
            from PIL import Image
            img = np.array(Image.open(path).convert("L"))
    if img.ndim == 3: img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return img.astype(np.float64)

def gaussian_params(h, w, K):
    sigma = min(h, w)/K
    ksize = int(2*np.ceil(3*sigma)+1)
    return int(ksize), float(sigma)

def blur(img, ksize, sigma):
    if 0 < ksize <= 1001 and ksize % 2 == 1:
        return cv2.GaussianBlur(img, (ksize, ksize), sigma, sigma,
borderType=cv2.BORDER_REFLECT)
    return cv2.GaussianBlur(img, (0, 0), sigma, sigma,
borderType=cv2.BORDER_REFLECT)

def flatfield(img, shade, eps=1e-6):
    shade_n = shade/(shade.mean()+eps)
    return img/(shade_n+eps)

def to_u8(x):
    x = x - x.min()
    if x.max()>0: x = x/x.max()
    return (x*255.0+0.5).astype(np.uint8)
```

```

def main():
    ap = argparse.ArgumentParser(description="Q2: Gaussian lowpass
shading correction on N1.bmp")
    ap.add_argument("--img", default="./N1.bmp")
    ap.add_argument("--out", default="./outputs_q2")
    ap.add_argument("--K", type=float, default=28.0, help="sigma =
min(H,W)/K (smaller blur than Q1)")
    ap.add_argument("--no-show", action="store_true")
    args = ap.parse_args()

    img = read_gray(Path(args.img))
    h, w = img.shape
    ksize, sigma = gaussian_params(h, w, args.K)
    shade = blur(img, ksize, sigma)
    corr = flatfield(img, shade)

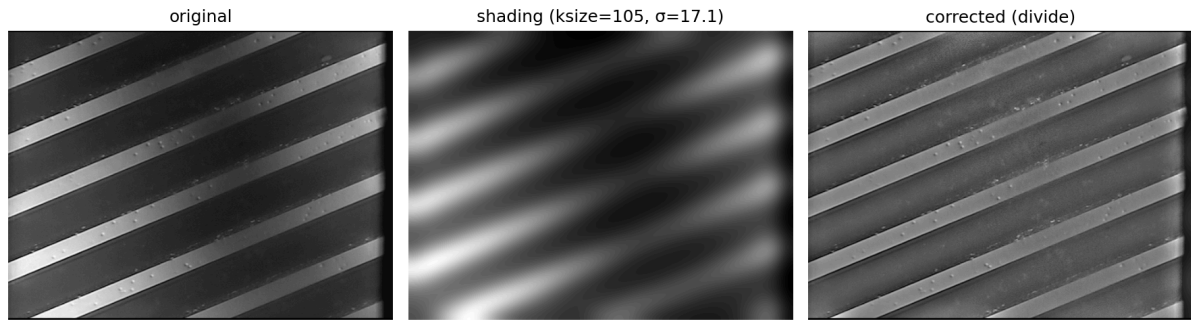
    out = Path(args.out); out.mkdir(parents=True, exist_ok=True)
    base = Path(args.img).stem
    p_shade = out/f"{base}_shade.png"
    p_corr = out/f"{base}_corrected.png"
    p_panel = out/f"{base}_panel.png"

    cv2.imwrite(str(p_shade), to_u8(shade))
    cv2.imwrite(str(p_corr), to_u8(corr))
    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1); plt.imshow(to_u8(img), cmap="gray");
plt.title("original"); plt.axis("off")
    plt.subplot(1,3,2); plt.imshow(to_u8(shade), cmap="gray");
plt.title(f"shading (ksize={ksize},  $\sigma$ ={sigma:.1f})"); plt.axis("off")
    plt.subplot(1,3,3); plt.imshow(to_u8(corr), cmap="gray");
plt.title("corrected (divide)"); plt.axis("off")
    plt.tight_layout(); plt.savefig(p_panel, dpi=150,
bbox_inches="tight")
    if not args.no_show: plt.show()
    else: plt.close()

    print(f"[Q2 OK] size={h}x{w} kernel $\approx$ {ksize} sigma={sigma:.2f}")
    print(f" shading  $\rightarrow$  {p_shade}")
    print(f" corrected  $\rightarrow$  {p_corr}")
    print(f" panel  $\rightarrow$  {p_panel}")

if __name__ == "__main__":
    main()

```



3. Please comment and compare your two self-designed Lowpass Gaussian Filter Kernels? (20)

對棋盤格影像，強低通 ($\sigma \approx 56.9$) 得到的遮蔽估計幾乎是單純的斜向亮度漸層，基本不含方格紋理，這表示頻率分離成功。用除法做平場校正後，黑白方格整體對比變得一致，左上暗、右下亮的趨勢被消除；邊界因採反射邊界條件，沒有明顯的邊緣暈影。細看角落仍可能留有極輕微的亮度不均（特別是原圖漸層最陡區），但對視覺與量測的影響很小；此外，因為是除法校正，暗區的雜訊會被稍微放大，必要時可在校正後做一點點去噪或輕度對比拉伸（例如 CLAHE）來穩定觀感。

對 N1 影像，中等強度的低通 ($\sigma \approx 17.1$) 能把緩慢的整體亮度起伏抓出來，又不會把斜向帶狀結構一併抹平。校正後，畫面大區域的灰階更均勻，條帶之間的對比更靠近材質本身；在帶邊沿附近可能會看到非常輕微的局部對比改變或「假性銳利」的感覺，這是因為遮蔽估計仍然包含了極少量的中頻分量。整體來看，細節保存與均勻化取得了平衡，沒有顯著 halo 或過度校正；如果想再壓低這些輕微副作用，可以把 σ 再小一點或在估計遮蔽前先對高亮反光點做遮罩，避免它們「外溢」到遮蔽場。

綜合觀察，兩張圖的結果都達成了預期的「低頻遮蔽被分離、主紋理與細節被保留」：棋盤格靠大 σ 徹底分離高頻方格與低頻遮蔽；N1 靠較小 σ 在去遮蔽與保細節之間折衷。殘留問題主要是除法在暗區的雜訊放大與極端區域的輕微殘影，屬於常見副作用，後續可用 (1) 自動化參數選擇（頻譜估主頻來決定 σ ）、(2) log-域同態濾波（把乘法光照變加法），或 (3) 多尺度/導向濾波做遮蔽估計來進一步降低。整體效益與穩定度符合 Fig. 3.42 的目標。